

08 - Test del software

Francesco Betti Sorbelli

Università degli studi di Perugia. `francesco.bettisorbelli@unipg.it`



Argomenti trattati

- 1 Test di sviluppo
- 2 Sviluppo guidato dai test
- 3 Test di rilascio
- 4 Test dell'utente

- I test hanno lo scopo di dimostrare che un programma fa quello per cui è stato pensato e di scoprire i difetti del programma prima che venga messo in uso.
- Quando si testa un SW, si esegue un programma utilizzando dati artificiali.
- Si controllano i risultati dell'esecuzione del test alla ricerca di errori, anomalie o informazioni sugli attributi non funzionali del programma.
- Può rivelare la presenza di errori, NON la loro assenza.
- I test fanno parte di un processo di verifica e validazione più generale, che comprende anche tecniche di validazione statica.

- Dimostrare allo sviluppatore e al cliente che il SW soddisfa i suoi requisiti.
 - Per il SW **personalizzato**, ciò significa che deve esserci almeno un test per ogni requisito del documento dei requisiti.
 - Per i prodotti SW **generici**, significa che devono essere previsti test per tutte le caratteristiche del sistema, e per le loro combinazioni, che saranno incorporate nel rilascio del prodotto.
- Scoprire situazioni in cui il comportamento del SW è scorretto, indesiderabile o non conforme alle sue specifiche.
 - I test sui difetti si occupano di eliminare comportamenti indesiderati del sistema, come crash, interazioni indesiderate con altri sistemi, calcoli errati e corruzione dei dati.

- Il primo obiettivo porta al test di validazione
 - Ci si aspetta che il sistema funzioni correttamente utilizzando un determinato insieme di casi di test che riflettono l'uso previsto del sistema.
- Il secondo obiettivo porta alla verifica dei difetti
 - I casi di test sono progettati per evidenziare i difetti. I casi di test nei test sui difetti possono essere deliberatamente oscuri e non devono necessariamente riflettere il modo in cui il sistema viene normalmente utilizzato.

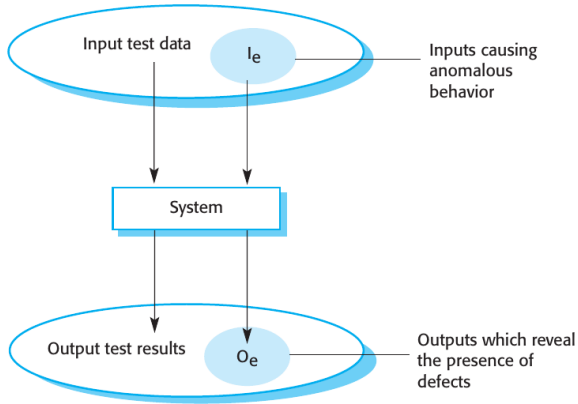
- Test di validazione

- Dimostrare allo sviluppatore e al cliente del sistema che il SW soddisfa i requisiti.
- Un test riuscito dimostra che il sistema funziona come previsto.

- Test dei difetti

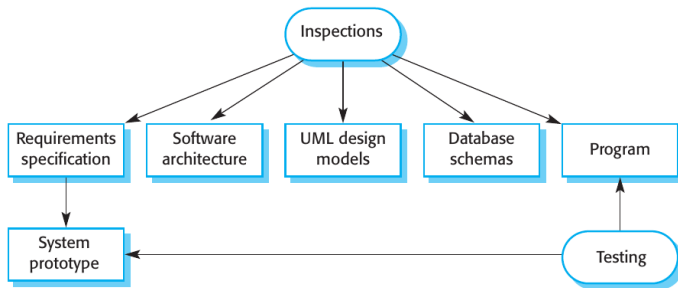
- Scoprire difetti o anomalie del SW, il cui comportamento non è corretto o non è conforme alle specifiche.
- Un test riuscito è un test che fa funzionare il sistema in modo errato e quindi espone un difetto del sistema.

Un modello di input-output per la verifica dei programmi



- L'obiettivo di V & V è quello di stabilire la fiducia che il sistema sia "adatto allo scopo".
- Dipende dallo scopo del sistema, dalle aspettative dell'utente e dall'ambiente di marketing.
 - Scopo del SW
 - Il livello di fiducia dipende dal grado di criticità del SW per un'organizzazione.
 - Aspettative degli utenti
 - Gli utenti possono avere basse aspettative nei confronti di alcuni tipi di SW.
 - Ambiente di marketing
 - Arrivare presto sul mercato può essere più importante che trovare difetti nel programma.

- **Ispezioni** del SW: si occupa dell'analisi della rappresentazione statica del sistema per scoprire i problemi (**verifica statica**)
 - Può essere integrato da strumenti di analisi dei documenti e del codice.
- **Test** del SW: si occupa di esercitare e osservare il comportamento del prodotto (**verifica dinamica**)
 - Il sistema viene eseguito con i dati di prova e viene osservato il suo comportamento operativo.



- Si tratta di esaminare la rappresentazione della fonte con l'obiettivo di scoprire anomalie e difetti.
- Le ispezioni non richiedono l'esecuzione di un sistema e possono quindi essere utilizzate prima dell'implementazione.
- Possono essere applicate a qualsiasi rappresentazione del sistema (requisiti, progetto, dati di configurazione, dati di test, ecc.)

- Durante il test, gli errori possono mascherare (nascondere) altri errori. Poiché l'ispezione è un processo statico, non è necessario preoccuparsi delle interazioni tra gli errori.
- Le versioni incomplete di un sistema possono essere controllate senza costi aggiuntivi. Se un programma è incompleto, è necessario sviluppare dei test specializzati per verificare le parti disponibili.
- Oltre a cercare i difetti del programma, un'ispezione può anche considerare attributi di qualità più ampi di un programma, come la conformità agli standard, la portabilità e la manutenibilità.

- Le ispezioni e i test sono tecniche di verifica complementari e non opposte.
- Entrambi dovrebbero essere utilizzati durante il processo V&V.
- Le ispezioni possono verificare la conformità a una specifica, ma non la conformità ai requisiti reali del cliente.
- Le ispezioni non possono verificare caratteristiche non funzionali come le prestazioni, l'usabilità, ecc.

- Test di **sviluppo**, in cui il sistema viene testato durante lo sviluppo per scoprire bug e difetti.
- Test di **rilascio**, in cui un team di collaudo separato testa una versione completa del sistema prima che venga rilasciata agli utenti.
- Test **utente**, in cui gli utenti o i potenziali utenti di un sistema lo testano nel proprio ambiente.

Argomenti trattati

- 1 Test di sviluppo
- 2 Sviluppo guidato dai test
- 3 Test di rilascio
- 4 Test dell'utente

Test di sviluppo

- Il test di sviluppo comprende tutte le attività di test svolte dal team che sviluppa il sistema.
 - Test di **unità**, in cui vengono testate singole unità di programma o classi di oggetti. I test di unità devono concentrarsi sulla verifica della funzionalità degli oggetti o dei metodi.
 - Test dei **componenti**, in cui diverse unità individuali vengono integrate per creare componenti compositi. Il test dei componenti deve concentrarsi sulla verifica delle interfacce dei componenti.
 - Test di **sistema**, in cui alcuni o tutti i componenti di un sistema sono integrati e il sistema viene testato nel suo complesso. Il test del sistema deve concentrarsi sulla verifica delle interazioni tra i componenti.

Test di unità

- Il test di unità è il processo di verifica dei singoli componenti in modo isolato.
- È un processo di verifica dei difetti.
- Le unità possono essere:
 - Singole funzioni o metodi all'interno di un oggetto
 - Classi di oggetti con diversi attributi e metodi
 - Componenti composti con interfacce definite per accedere alle loro funzionalità.

Test delle classi di oggetti

- La copertura completa dei test di una classe comporta
 - Verifica di tutte le operazioni associate a un oggetto
 - Impostazione e interrogazione di tutti gli attributi degli oggetti
 - Esercitare l'oggetto in tutti gli stati possibili.
- L'ereditarietà rende più difficile la progettazione di test di classi di oggetti, poiché le informazioni da testare non sono localizzate.

Test automatizzati

- Quando possibile, i test di unità devono essere automatizzati, in modo che i test vengano eseguiti e controllati senza intervento manuale.
- Nei test di unità automatizzati, si utilizza un framework di automazione dei test (come JUnit) per scrivere ed eseguire i test del programma.
- I framework per i test di unità forniscono classi di test generiche che vengono estese per creare casi di test specifici. Possono quindi eseguire tutti i test che avete implementato e riportare, spesso attraverso una GUI, il successo di altri test.

Componenti di test automatizzati

- Una parte di configurazione, in cui si inizializza il sistema con il caso di test, ovvero gli input e gli output previsti.
- Una parte di chiamata, in cui si chiama l'oggetto o il metodo da testare.
- Una parte dell'asserzione in cui si confronta il risultato della chiamata con il risultato atteso. Se l'asserzione è vera, il test ha avuto successo, se è falsa, il test è fallito.

Scelta dei casi di test di unità

- I casi di test devono dimostrare che, quando viene utilizzato come previsto, il componente che si sta testando fa ciò che si suppone faccia.
- Se ci sono difetti nel componente, questi dovrebbero essere rivelati dai casi di test.
- Questo porta a due tipi di casi di test di unità:
 - Il primo di questi dovrebbe riflettere il normale funzionamento di un programma e mostrare che il componente funziona come previsto.
 - L'altro tipo di caso di test dovrebbe basarsi sull'esperienza di test per capire dove si verificano i problemi più comuni. Dovrebbe utilizzare input anomali per verificare che questi vengano elaborati correttamente e non mandino in crash il componente.

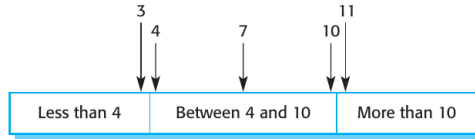
Strategie di test

- Test di **partizione**, in cui si identificano gruppi di input che hanno caratteristiche comuni e devono essere elaborati nello stesso modo.
 - Dovete scegliere i test all'interno di ciascuno di questi gruppi.
- Test basati su **linee guida**, in cui si utilizzano linee guida per scegliere i casi di test.
 - Queste linee guida riflettono l'esperienza precedente sui tipi di errori che i programmatori spesso commettono durante lo sviluppo dei componenti.

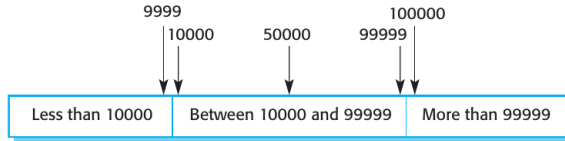
Test di partizione

- I dati di input e i risultati di output spesso rientrano in classi diverse, dove tutti i membri di una classe sono correlati.
- Ognuna di queste classi è una **partizione di equivalenza** o dominio in cui il programma si comporta in modo equivalente per ogni membro della classe.
- I casi di test devono essere scelti da ogni partizione.

Partizioni di equivalenza



Number of input values



Input values

Linee guida per i test (sequenze, liste, array)

- Testate il SW con sequenze che hanno un solo valore.
- Utilizzare sequenze di dimensioni diverse in test diversi.
- Derivare i test in modo che si acceda al primo, al centro e all'ultimo elemento della sequenza.
- Test con sequenze di lunghezza zero.

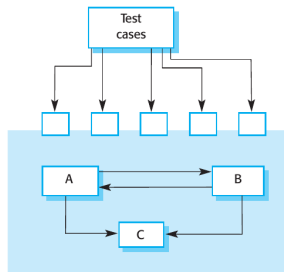
Linee guida generali per i test

- Scegliere gli input che costringono il sistema a generare tutti i messaggi di errore.
- Progettare input che causano l'overflow dei buffer di ingresso
- Ripetere più volte lo stesso input o la stessa serie di input
- Forzare la generazione di uscite non valide
- Forzare i risultati dei calcoli in modo che siano troppo grandi o troppo piccoli.

Test dei componenti

- I componenti SW sono spesso componenti compositi, costituiti da diversi oggetti che interagiscono tra loro: macro-servizi formati da micro-servizi.
- Si accede alle funzionalità di questi oggetti attraverso l'interfaccia del componente definito.
- Il test dei componenti compositi deve quindi concentrarsi sulla dimostrazione che l'interfaccia del componente si comporta secondo le sue specifiche.
 - Si può presumere che i test di unità sui singoli oggetti del componente siano stati completati.

Test dell'interfaccia



- L'obiettivo è rilevare i guasti dovuti a errori di interfaccia o a ipotesi non valide sulle interfacce.
- Tipi di interfaccia
 - Interfacce di parametri: Dati passati da un metodo o da una procedura a un'altra.
 - Interfacce di memoria condivisa: Un blocco di memoria è condiviso tra procedure o funzioni.
 - Interfacce procedurali: Il sottosistema incapsula un insieme di procedure che possono essere richiamate da altri sottosistemi.
 - Interfacce per il passaggio di messaggi: I sottosistemi richiedono servizi ad altri sottosistemi.

Errori di interfaccia

- Uso improprio dell'interfaccia
 - Un componente chiamante chiama un altro componente e commette un errore nell'uso della sua interfaccia, ad esempio parametri nell'ordine sbagliato.
- Incomprensione dell'interfaccia
 - Un componente chiamante incorpora ipotesi sul comportamento del componente chiamato che non sono corrette.
- Errori di temporizzazione
 - A real-time, il componente chiamato e quello chiamante operano a velocità diverse e si accede a informazioni non aggiornate.

Linee guida per il test dell'interfaccia

- Progettate i test in modo che i parametri di una procedura chiamata si trovino agli estremi del loro intervallo.
- Testare sempre i parametri dei puntatori con puntatori nulli.
- Progettare test che causino il fallimento del componente.
- Utilizzare lo stress test nei sistemi di passaggio di messaggi.
- Nei sistemi a memoria condivisa, variare l'ordine di attivazione dei componenti.

Test del sistema

- Il collaudo del sistema durante lo sviluppo prevede l'integrazione dei componenti per creare una versione del sistema e quindi il collaudo del sistema integrato.
- L'obiettivo del test di sistema è verificare le interazioni tra i componenti.
- Il test del sistema verifica che i componenti siano compatibili, interagiscano correttamente e trasferiscano i dati giusti al momento giusto attraverso le loro interfacce.
- I test di sistema verificano il comportamento emergente di un sistema.

Test del sistema e dei componenti

- Durante il collaudo del sistema, i componenti riutilizzabili che sono stati sviluppati separatamente e i sistemi off-the-shelf possono essere integrati con componenti di nuova concezione. Il sistema completo viene quindi testato.
- I componenti sviluppati da diversi membri del team o da sotto-team possono essere integrati in questa fase. Il collaudo del sistema è un processo collettivo piuttosto che individuale.
 - In alcune aziende, il collaudo del sistema può coinvolgere un team di collaudo separato, senza alcun coinvolgimento di progettisti e programmatori.

Politiche di test

- Un test esaustivo del sistema è impossibile, quindi si possono sviluppare politiche di test che definiscono la copertura di test del sistema richiesta.
- Esempi di politiche di verifica:
 - Tutte le funzioni del sistema a cui si accede attraverso i menu devono essere testate.
 - È necessario testare le combinazioni di funzioni (ad esempio, la formattazione del testo) a cui si accede attraverso lo stesso menu.
 - Quando è previsto l'inserimento di dati da parte dell'utente, tutte le funzioni devono essere testate sia con l'inserimento corretto che con quello errato.

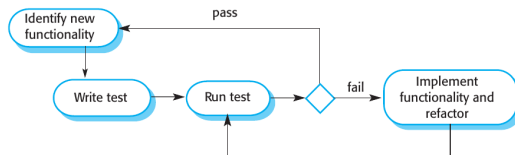
Argomenti trattati

- 1 Test di sviluppo
- 2 Sviluppo guidato dai test
- 3 Test di rilascio
- 4 Test dell'utente

Sviluppo guidato dai test

- Lo sviluppo guidato dai test è un approccio allo sviluppo dei programmi in cui si intersecano i test e lo sviluppo del codice.
- I test vengono scritti prima del codice e il "superamento" dei test è il fattore critico dello sviluppo.
- Si sviluppa il codice in modo incrementale, insieme a un test per quell'incremento. Non si passa all'incremento successivo finché il codice sviluppato non supera il test.
- È stato introdotto come parte di metodi agili come l'Extreme Programming. Tuttavia, può essere utilizzato anche nei processi di sviluppo pianificati.

Attività del processo dello sviluppo guidato dai test



- Iniziate identificando l'incremento di funzionalità richiesto. Di solito dovrebbe essere piccolo e implementabile in poche righe di codice.
- Scrivere un test per questa funzionalità e implementarlo come test automatico.
- Eseguire il test, insieme a tutti gli altri test che sono stati implementati. Inizialmente, la funzionalità non è stata implementata, quindi il nuovo test fallirà.
- Implementare la funzionalità e rieseguire il test.
- Una volta che tutti i test sono stati eseguiti con successo, si passa all'implementazione della funzionalità successiva.

Vantaggi dello sviluppo guidato dai test

- Copertura del codice
 - Ogni segmento di codice scritto ha almeno un test associato, quindi tutto il codice scritto ha almeno un test.
- Test di regressione
 - Una suite di test di regressione viene sviluppata in modo incrementale durante lo sviluppo di un programma.
- Debug semplificato
 - Quando un test fallisce, dovrebbe essere ovvio dove si trova il problema. Il codice appena scritto deve essere controllato e modificato.
- Documentazione del sistema
 - I test stessi sono una forma di documentazione che descrive ciò che il codice dovrebbe fare.

Test di regressione

- Il test di regressione consiste nel testare il sistema per verificare che le modifiche non abbiano "rotto" il codice precedentemente funzionante.
- In un processo di test manuale, il test di regressione è costoso, ma con i test automatizzati è semplice e diretto. Tutti i test vengono rieseguiti ogni volta che viene apportata una modifica al programma.
- I test devono essere eseguiti "con successo" prima che la modifica venga impegnata.

Argomenti trattati

- 1 Test di sviluppo
- 2 Sviluppo guidato dai test
- 3 Test di rilascio
- 4 Test dell'utente

Test di rilascio

- Il test di rilascio è il processo di verifica di una particolare release di un sistema destinata all'uso al di fuori del team di sviluppo.
- L'obiettivo principale del processo di test di rilascio è quello di convincere il fornitore del sistema che è sufficientemente buono per essere utilizzato.
 - I test di rilascio, quindi, devono dimostrare che il sistema offre le funzionalità, le prestazioni e l'affidabilità specificate e che non si guasta durante il normale utilizzo.
- Il test di rilascio è di solito un processo di test black-box in cui i test derivano solo dalle specifiche del sistema.

Test di rilascio e test di sistema

- Il test di rilascio è una forma di test di sistema.
- Differenze importanti:
 - Un team separato, non coinvolto nello sviluppo del sistema, dovrebbe essere responsabile del test di rilascio.
 - Il test del sistema da parte del team di sviluppo deve concentrarsi sulla scoperta di bug nel sistema (test dei difetti). L'obiettivo del test di rilascio è verificare che il sistema soddisfi i requisiti e sia sufficientemente valido per l'uso esterno (test di validazione).
- Il test basato sui requisiti prevede l'esame di ogni requisito e lo sviluppo di uno o più test per esso.

Test delle prestazioni

- Una parte dei test di rilascio può comprendere la verifica delle proprietà emergenti di un sistema, come le prestazioni e l'affidabilità.
- I test devono riflettere il profilo di utilizzo del sistema.
- I test sulle prestazioni di solito prevedono la pianificazione di una serie di test in cui il carico viene aumentato costantemente fino a quando le prestazioni del sistema diventano inaccettabili.
- Lo stress test è una forma di verifica delle prestazioni in cui il sistema viene deliberatamente sovraccaricato per testare il suo comportamento in caso di guasto.

Argomenti trattati

- 1 Test di sviluppo
- 2 Sviluppo guidato dai test
- 3 Test di rilascio
- 4 Test dell'utente

Test dell'utente

- Il collaudo da parte dell'utente o del cliente è una fase del processo di collaudo in cui gli utenti o i clienti forniscono input e consigli sul collaudo del sistema.
- Il test degli utenti è essenziale, anche quando sono stati eseguiti test completi del sistema e del rilascio.
 - Il motivo è che le influenze dell'ambiente di lavoro dell'utente hanno un effetto importante sull'affidabilità, le prestazioni, l'usabilità e la robustezza di un sistema. Questi fattori non possono essere replicati in un ambiente di test.

Tipi di test utente

- Test alfa
 - Gli utenti del SW collaborano con il team di sviluppo per testare il SW presso la sede dello sviluppatore.
- Test beta
 - Una release del SW viene messa a disposizione degli utenti per consentire loro di sperimentare e di segnalare agli sviluppatori del sistema i problemi riscontrati.
- Test di accettazione
 - I clienti testano un sistema per decidere se è pronto o meno per essere accettato dagli sviluppatori del sistema e distribuito nell'ambiente del cliente. Principalmente per i sistemi personalizzati.